

UFCFUR-15-3 - Advanced Artificial Intelligence

Zoe Haines – 21033821

Bristol Regional Foot Network – AI Image Classification

<https://github.com/goetic-zoe/UFCFUR-15-3-Advanced-Artificial-Intelligence/tree/master>

Introduction.....	2
Initial Challenges	3
Dataset	4
Data Examples.....	4
Data Augmentation	5
Model Comparison.....	6
Support Vector Machines (SVMs)	6
Convolutional Neural Networks (CNNs)	7
Comparison Conclusion.....	7
The Model.....	7
Model Architecture.....	8
Embedded Augmentation	8
Feature Extraction.....	8
Final Classification and Regularisation	8
Model Compilation and Complexity.....	9
Results and Evaluation	9
Use of Generative AI	11
Legal, Ethical, Social and Professional (LESP) Implications	11
References	12

Introduction

The Bristol Regional Food Network aims to support local food producers by improving the way fruit and vegetable quality is assessed before products are sold or distributed. To address this, the organisation is exploring the use of Artificial Intelligence (AI) and computer vision techniques to automate quality assessment.

Within this technical report it documents the creation and applications of these techniques using a fresh and rotten/stale fruit and vegetable classification dataset to, in the end build, train and use a custom, purpose built, convolutional neural network (CNN). This purpose-built CNN will be able to accurately assess and grade fruit and vegetables giving them a letter grading and a score of its confidence along with other statistics.

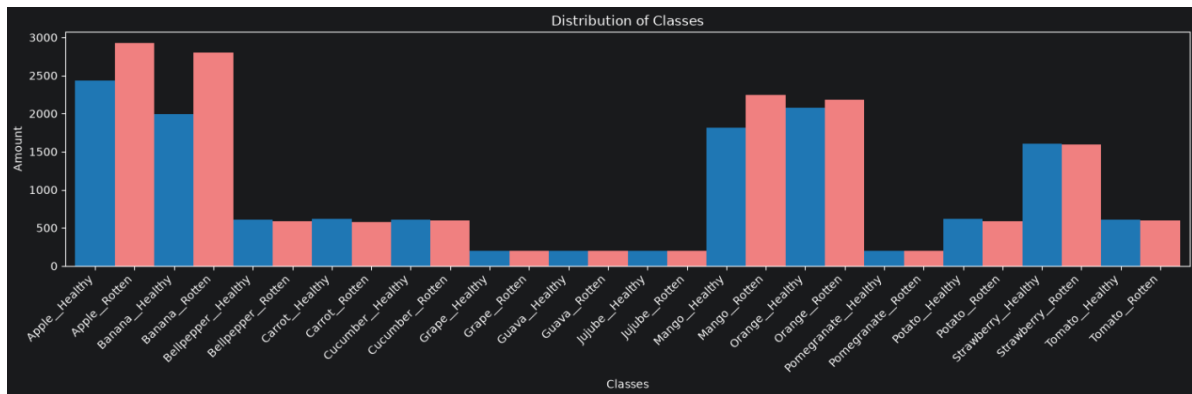
This system was designed and developed by Zoe Haines (21033821) for the University of the West of England, Bristol. This technical report will cover design, options for implementation and a full evaluation of the system with legal, ethical, social and professional (LESP) considerations kept at a high priority throughout the process. This document will feature many visualisations to keep the process explainable and responsible, allowing readers to digest the information clearly.

The full repository for this system is available [here](https://github.com/goetic-zoe/UFCFUR-15-3-Advanced-Artificial-Intelligence/tree/master) (<https://github.com/goetic-zoe/UFCFUR-15-3-Advanced-Artificial-Intelligence/tree/master>)

Initial Challenges

When developing a well performing image classification AI the dataset used to train it is paramount as it affects how targeted or generalised the AI is and it can leave weaknesses within the AI that can cause it to produce incorrect predictions, lower processing speeds or overall worse performance than expected.

The dataset used as instructed was the **Fresh and Rotten/Stale Fruits and Vegetables Classification Dataset** (Subhan, M. 2024). Despite this being a larger dataset along with 43,000 views and just under 1000 downloads it has some distribution issues where there are many more images of certain classes within the dataset leaving the distribution that the CNN will be trained on somewhat biased.



As shown the dataset has a lot more images instances of apples, bananas, mangos, oranges and strawberries luckily these fruits/vegetables are more common than some of the others with less images, this will need to be taken into account when preprocessing the data and training the CNN.

Another challenge that could be identified from the offset, allowing it to be addressed, for quality assessment fresh produce can vary in colour and appearance within the same grading bracket due to differing plant genetics, this can be somewhat resolved by designing the training regime of the model to reinforce generalised visual feature learning rather than just memorising all the training images. It is also addressed when selecting the split of training, validation and testing data as if the model is trained on a higher percentage of the data, it is more likely to have a generalised approach as it will encounter more variation than if not. This is why an 80/10/10 data split was chosen for this model's training.

Dataset

The dataset consists of 29.3k images and with the previously stated 80/10/10 split it leaves us with these values for the training, validation and testing data.

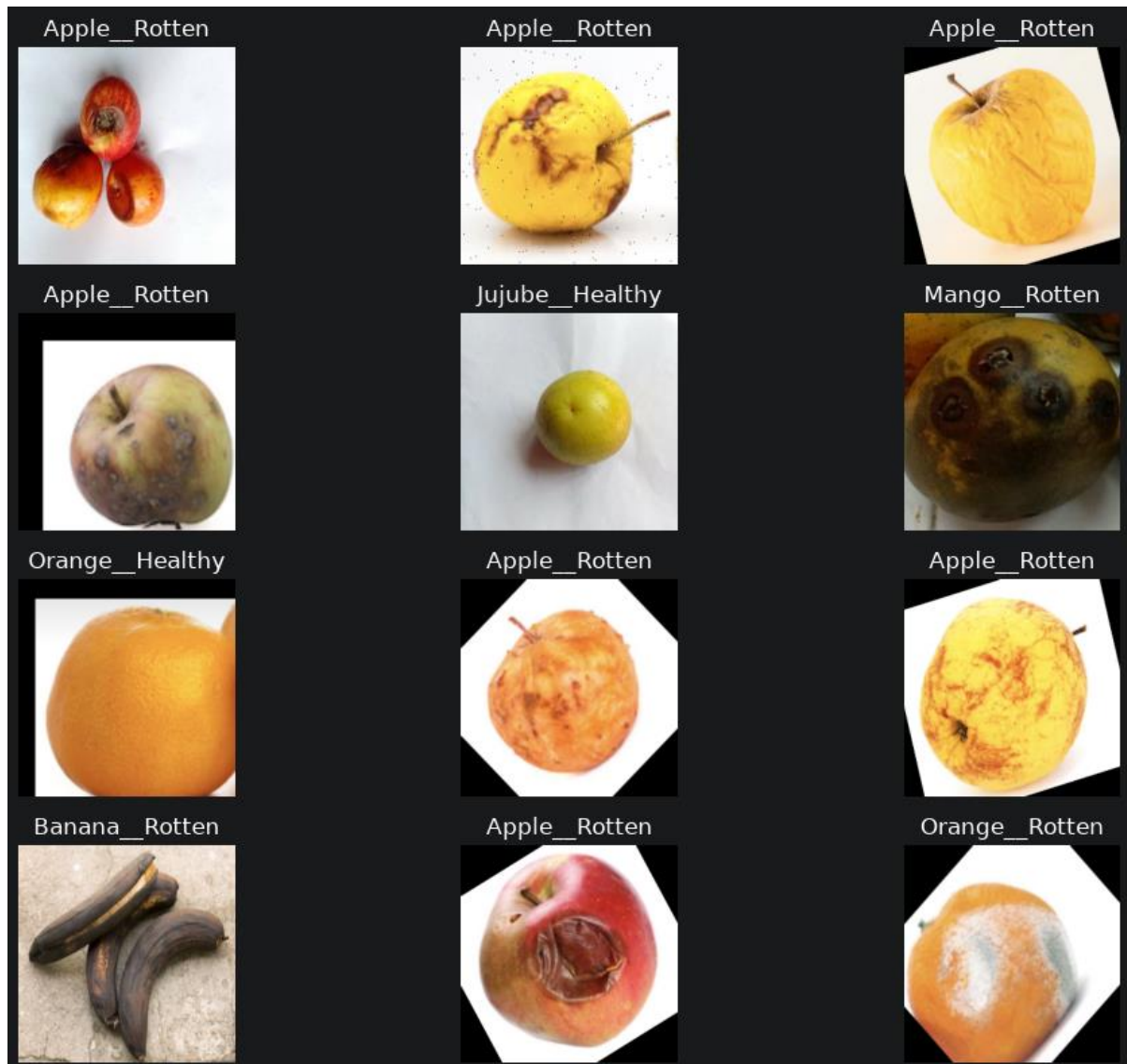
```
Training Data: 23418
Validation Data: 2927
Testing Data: 2928
```

With the stated distribution of the classes and there being many classes which may be underrepresented a library like 'Augmenter' could be used to create more data by augmenting and modifying pre-existing data so that it is somewhat different than the original however in this case due to multiple concerns including data storage worries, as the dataset is over 5GB, and by flooding the model with very similar images only slightly changed it was chosen not to be used however if this model was to be trained on a smaller dataset but with a similar class distribution this could be a powerful solution to mitigating these issues.

Data augmentation was used to vary the data on each epoch, and all the data is shuffled throughout. This should create a more generalised system that can recognise and classify more images accurately especially those that it has not seen before from its training regime. This will be expanded on more thoroughly later within this document giving a full explanation and break down of why specific augmentation was used and why others were not used.

Data Examples

Provided are some examples of the dataset that the model is going to be trained on, this allows you to understand the types of images and data expected to be passed into this model.



As shown the model is going to be trained to expect images of fruits and vegetables in different states of fresh and rotten/stale from different orientations. The orientation changes with rotated images are baked into the dataset to make it more varied and usable for creating more generalised image recognition systems, because the orientation changes are incorporated within the dataset itself it means that these examples are before our own data augmentation which can change the orientation of the images as well however as it does this every epoch, it adds more variety to the already varied images within the dataset.

Data Augmentation

This system uses simplistic data augmentation at each epoch, while it could also include random shearing, brightness changes, colour changes along with many other options, it only uses randomised zooming, rotation and horizontal flipping. This was chosen to add a level of augmentation and changing how the data is presented to the model but to not over-complicate it due to the use-case of this system means that certain aspects like shearing should not happen when implemented correctly, in the same vain random colour changes were determined to not be viable due to the nature of this system as a rotten/stale classifier which for the vast majority of fruits are changes in colour and this was determined to possibly compromise the learning of the model through data poisoning, this is for the same reason random changes in brightness are not used as it could cause the same outcomes and it is assumed that this system for classification would be installed in a well-lit environment.

Randomised zooming, rotation and flipping were used as when the model is in production and being used, the produce being processed by the system will be at different rotations, distance to the camera and orientations and the system needs to be able to identify the produce and determine if it is rotten or stale no matter these differences.

The exact variables used for the randomised zooming, rotation and flipping are shown below.

```
RandomZoom(0.1),  
RandomRotation(0.1),  
RandomFlip('horizontal')
```

Model Comparison

For the development of this system two different types of machine learning models were compared; convolutional neural networks (CNNs) and support vector machines (SVMs). Both models can be used for image classification however they have different use-cases and generally CNNs provide better performance over SVMs however this is not always the case. After comparison a CNN was selected for this system due to it being vastly superior for the specific circumstances as will be laid out.

Support Vector Machines (SVMs)

SVMs when used for image classification are shallow classifiers maximizing hyperplanes which requires manually crafted feature engineering. In image classification, SVMs are utilised when you require very low training and storage demands as they require very little hardware and can be trained relatively fast depending on how many features are required to be implemented.

Furthermore, SVMs are exceptionally useful when the resource constraints are high as it does not require dedicated GPUs and are ideal for low-power devices such as microcontrollers. However, their need of specialised predefined image features can make them more difficult to implement if you do not have this data and must be implemented from scratch, expanding on this it can cause SVMs to struggle analysing complex objects which in a system like this could cause accuracy issues (Wang et al., 2020).

SVMs are more useful when combined with small data sizes and therefore small datasets where you have hundreds of images where large and deep neural networks would instantly overfit the data causing it to lose accuracy when tasked with data it has not encountered before, however in this system the data set is of tens of thousands of individual pieces of data which could critically dampen the use of an SVM.

Convolutional Neural Networks (CNNs)

CNNs are deep neural networks that utilise convolutions within its training which automatically extracts hierarchical spatial features with a goal to solve gradient descent for minimising error loss.

A major downside of CNNs is that they are very hardware intensive models to train requiring large amounts of storage space themselves and typically requires dedicated GPUs for acceleration of training that could take multiple days without. Another downside is that the datasets typically used for CNNs are very large taking up multiple gigabytes possibly hundreds on occasion however for this system it creates an advantage that being CNNs perform better on larger datasets as they are less likely to overfit due to having more data points and as our dataset contains roughly twenty-five thousand images a CNN could be ideal for this model as the data size is more relevant for CNNs than other types of model as it scales efficiently and improves its accuracy with more data.

CNNs can also achieve high accuracy values, which will be shown by this system, which can achieve near human or state of the art accuracy in multi-class visualisation tasks which in a business setting, like this system, is a major factor to consider. CNNs are also better at recognising and processing complex objects where there are more subtle differences in lighting and textures. (Patil et al., 2024)

Comparison Conclusion

After considering the pros and cons of both SVMs and CNNs it was determined that a CNN would be more applicable for this system both due to its higher accuracy and preference towards larger datasets. It was also decided that a CNN would be better due to not having limited access to hardware to train a CNN as this system utilised Nvidia GPUs during its training.

The Model

Based on the result of the previous comparison, a CNN was selected and implemented using the TensorFlow and Keras libraries. It is built on top of the Keras 'Sequential' API which enables the use of linear stacks of layers making it an ideal choice for this image classification task.

The model architecture is designed to handle the previously identified challenges balancing deep feature extraction and regularisation methods to prevent overfitting. The model processes 244x244 RGB images.

Model Architecture

Embedded Augmentation

To possibly mitigate the previously stated challenges and variance in produce, data augmentation (RandomZoom, RandomRotation and RandomFlip) is embedded directly within the model as stated before. This technique makes sure that the network never sees the exact same image across epochs.

Feature Extraction

The network uses four convolutional blocks.

- Convolutional Layers (Conv2D): Four layers with increasing filter count (32, 64, 128 and 256) extract elements from basic edges to complex textures. Each uses a 3x3 kernel and preserves spatial dimensions via padding using ReLU activation.
- Pooling Layers (MaxPooling2D): A 2x2 pooling layer is after each convolution which reduces spatial dimensions. This reduction lowers computational load and makes feature detection translationally invariant.

Final Classification and Regularisation

- Flatten and Dense Layers: 3D maps are flattened into 1D vectors and fed into a 256-neuron dense layer (ReLU).
- Dropout: A 0.5 dropout rate prevents overfitting due to the imbalanced distribution of data.
- Output: The final dense layer matches the class count and uses a softmax activation to output a probability distribution and confidence score.

A full summery of the architecture of the model is shown below.

Layer (type)	Output Shape	Param #
sequential_2 (Sequential)	(None, 244, 244, 3)	0
conv2d_4 (Conv2D)	(None, 244, 244, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 122, 122, 32)	0
conv2d_5 (Conv2D)	(None, 122, 122, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 61, 61, 64)	0
conv2d_6 (Conv2D)	(None, 61, 61, 128)	73,856
max_pooling2d_6 (MaxPooling2D)	(None, 30, 30, 128)	0
conv2d_7 (Conv2D)	(None, 30, 30, 256)	295,168
max_pooling2d_7 (MaxPooling2D)	(None, 15, 15, 256)	0
flatten_1 (Flatten)	(None, 57600)	0
dense_2 (Dense)	(None, 256)	14,745,856
dropout_1 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 28)	7,196

Total params: 15,141,468 (57.76 MB)
Trainable params: 15,141,468 (57.76 MB)
Non-trainable params: 0 (0.00 B)

Model Compilation and Complexity

The model uses the Adam optimiser, sparse categorical crossentropy loss and tracks accuracy.

It totals to 15,141,468 trainable parameters of which over 14.7 million of these connect the flatten and first dense layers creating a high computational capacity to accurately classify visual differences between fresh and stale produce.

Results and Evaluation

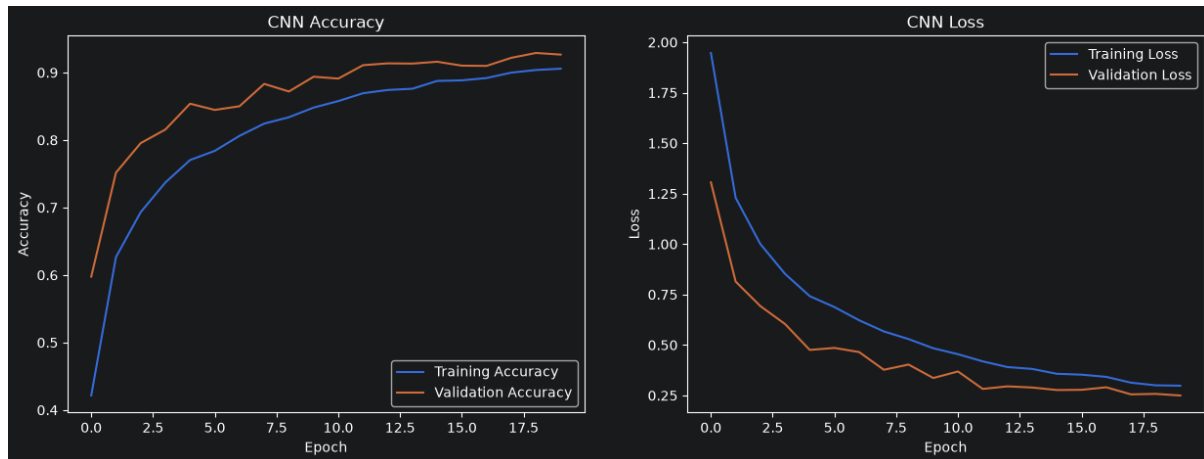
The proposed model, previously detailed, went through training and evaluation showing very good performance in classifying fresh and rotten produce.

```

Epoch 15/20
732/732 — 267s 363ms/step - accuracy: 0.8875 - loss: 0.3573 - val_accuracy: 0.9160 - val_loss: 0.2768
Epoch 16/20
732/732 — 265s 361ms/step - accuracy: 0.8884 - loss: 0.3529 - val_accuracy: 0.9101 - val_loss: 0.2778
Epoch 17/20
732/732 — 268s 366ms/step - accuracy: 0.8918 - loss: 0.3420 - val_accuracy: 0.9098 - val_loss: 0.2903
Epoch 18/20
732/732 — 276s 376ms/step - accuracy: 0.8998 - loss: 0.3133 - val_accuracy: 0.9218 - val_loss: 0.2553
Epoch 19/20
732/732 — 270s 367ms/step - accuracy: 0.9037 - loss: 0.3001 - val_accuracy: 0.9289 - val_loss: 0.2577
Epoch 20/20
732/732 — 266s 362ms/step - accuracy: 0.9057 - loss: 0.2983 - val_accuracy: 0.9265 - val_loss: 0.2496
CNN Testing Data Loss: 0.22730693221092224 ,
CNN Testing Data Accuracy: 0.936133861541748
  
```

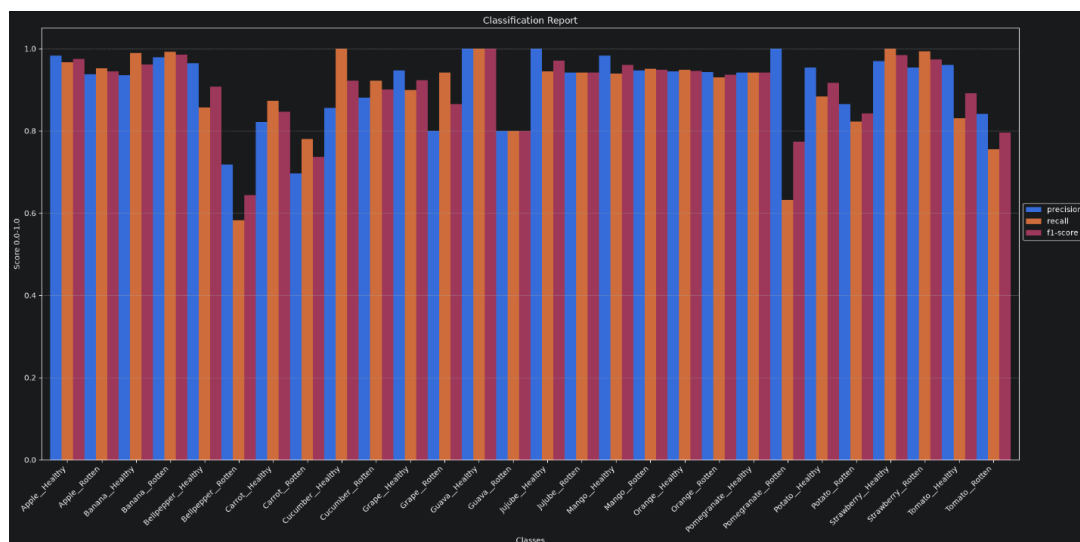
As shown from the execution logs of training from the final 5 epochs and running the testing data through the model it achieved approximately 92-93% accuracy with a minimal testing loss of 0.227

The stability of the training is visualised in model performance graphs.

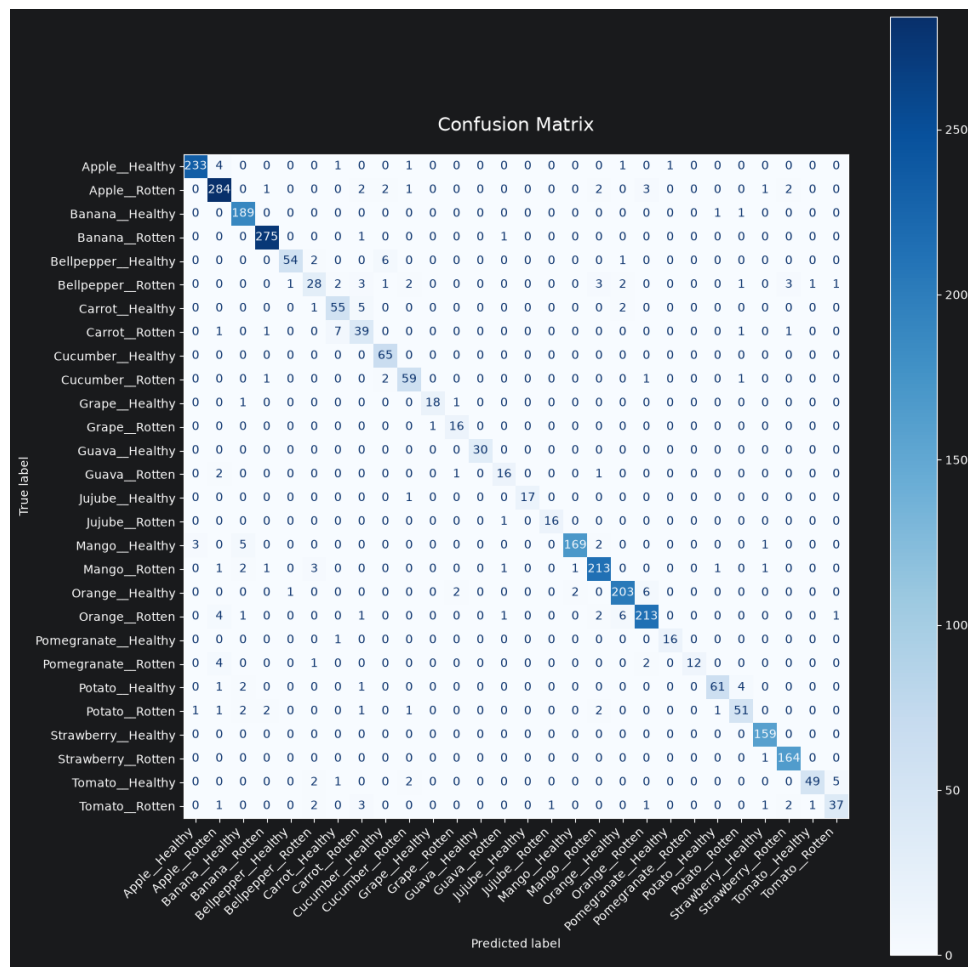


Both curves for training and validation accuracy and loss converge without any major deviance. This shows that the embedded data augmentation and regularisation correctly mitigated overfitting which allows the model to generalise well.

The classification report shows the model's success by displaying consistently high precision, recall and F1-scores across most classes.



This good multi-class accuracy is reinforced by the confusion matrix that displays a distinct, strong diagonal tend which shows that there are few misclassifications demonstrating that the network can reliably tell between fresh and stale/rotten produce without confusing visually similar produce.



To conclude and test the system's real-world utility a single-image prediction test was performed, the system is fed in an image of a healthy grape that it has never seen before and accurately predicts the correct class with a 95% confidence score. It correctly converts the raw classification into a quality assessment that outputs calculated scores for different attributes.

```

1/1 ————— 0s 45ms/step
Produce Type: Grape
Freshness Label: Healthy
Predicted Class: Grape_Healthy
Confidence: 0.9504
Colour score: 94
Size score: 94
Ripeness score: 89
Grade: A

```

Use of Generative AI

Generative AI was used in the production of this system occasionally mainly being used to remove repetitive development, using it for tasks such as creating matplotlib graphs and simple algorithms that were required multiple times but with only slight changes in-between them to improve workflow.

Legal, Ethical, Social and Professional (LESP) Implications

The use of this classification system introduces multiple LESP implications. Legally, the system must comply with regional food trading standards aligning the grading method. Ethically, mitigating the dataset imbalance is major as it could unfairly punish some producers by misjudging their produce. Socially, while a system like this can streamline assessment operations and can reduce food waste, it changes local labour dynamics by replacing manual graders. Professionally, the system was developed to transparent standards ensuring the system remains explainable, responsible and well documented.

References

- Subhan, M. (2024). *Fruit and Vegetable Disease (Healthy vs Rotten)*. Kaggle.com.
<https://www.kaggle.com/datasets/muhammad0subhan/fruit-and-vegetable-disease-healthy-vs-rotten>
- Wang, P., Fan, E., & Wang, P. (2020). Comparative Analysis of Image Classification Algorithms Based on Traditional Machine Learning and Deep Learning. *Pattern Recognition Letters*, 141, 61–67. <https://doi.org/10.1016/j.patrec.2020.07.042>
- Patil, A. P., Mahesh, C., Agarwal, A., & Areeckal, A. S. (2024). Comparative Analysis of Support Vector Machine and Convolutional Neural Network for Image Classification. *2024 Third International Conference on Artificial Intelligence, Computational Electronics and Communication System (AICECS)*, 1–4.
<https://doi.org/10.1109/aicecs63354.2024.10956324>